Documentación del proyecto – Grupo 4 · Reto 1

Foro interno de preguntas y respuestas para trabajadores e ingenieros (Aergibide SL)

Equipo: Zaira, Iker y Sara

Repositorio: github.com/IkerPoza/Grupo4Reto1

1. Objetivos del proyecto

Objetivo general. Desarrollar una aplicación web que permita a los trabajadores de la empresa plantear dudas técnicas y resolverlas entre compañeros, de forma similar a un foro tipo Stack Overflow, pero de uso interno. El encargo es de Aergibide SL, empresa del sector aeronáutico cuyo departamento de I+D+I cuenta con más de 150 ingenieros que trabajan de forma muy autónoma y desconocen qué compañeros ya se han enfrentado a sus mismos problemas o son expertos en el tema.

Módulos implicados: 0612 Desarrollo web en entorno cliente, 0613 Desarrollo web en entorno servidor, 0614 Despliegue de aplicaciones web, 0615 Diseño de interfaces web, Desarrollo de back-end avanzado y Desarrollo de aplicaciones full stack. Grupos de 3 personas, 42 días.

Objetivos específicos.

- Permitir el registro y el inicio de sesión de usuarios, diferenciando dos roles: usuario y administrador.
- Permitir publicar preguntas clasificadas por tema, con imagen opcional.
- Permitir responder a las preguntas (y a otras respuestas), adjuntando archivos.
- Valorar la calidad de las respuestas mediante votos positivos y negativos.
- Facilitar la búsqueda y la organización del contenido: búsqueda por título, filtro por tema, preguntas guardadas y ordenación.
- Ofrecer a cada usuario un perfil con su información y sus estadísticas de participación.
- Dar al administrador herramientas de moderación (borrar preguntas, respuestas y usuarios) y de gestión de usuarios.
- Aplicar una arquitectura MVC en PHP con acceso a datos mediante PDO y una interfaz adaptable a distintos tamaños de pantalla.

2. Planificación

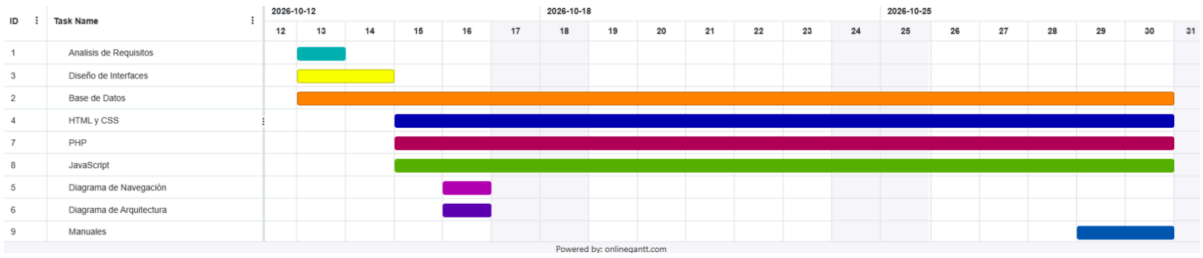
2.1. Tareas a realizar

Nº	Paso del enunciado	Trabajo realizado en el proyecto
1	Análisis y estudio del reto	Identificación de las funcionalidades mínimas, los roles y los datos necesarios.
2	Diseño de la base de datos	Modelo entidad-relación y script SQL (7 tablas).

Nº	Paso del enunciado	Trabajo realizado en el proyecto
3	Diseño de la interfaz de usuario y UX	Prototipos, diagrama de navegación y diagrama de arquitectura.
4	Lógica de presentación	Vistas PHP, layout común, hojas de estilo, tipografía, iconos SVG y diseño adaptable.
5	Tratamiento de datos en el cliente	Validaciones en JavaScript con expresiones regulares y mensajes de error junto a cada campo.
6	Envío de datos al servidor	Formularios POST (con archivos) y Fetch API con JSON para los votos.
7	Tratamiento de datos en el servidor	Router, controladores, modelos con PDO, sesiones, subida de archivos y permisos por rol.
8	Despliegue de la aplicación	[COMPLETAR] Ver el apartado 5.
9	Documentación	Esta memoria, el manual de usuario y el documento de seguimiento diario.
10	Presentación	Exposición el 4/11 y defensa el 5 y 6/11.

2.2. Planificación temporal

El reto dura 42 días: la parte autónoma empieza el viernes 9/10/2026, la entrega es el martes 3/11/2026, la exposición el miércoles 4/11 y las defensas el jueves 5 y el viernes 6/11.



Fechas	Trabajo realizado
14–16 oct	Imágenes y layout, vista de usuario, CSS de cabecera y pie, tipografía, vista de preguntas y primeros scripts JS.
16–20 oct	Modelo y controlador de usuario, obtención de preguntas, conexión de las vistas con el layout, funcionalidades de usuario y cierre de sesión.
21–22 oct	Comentarios en el código, perfil casi terminado, detalles de preguntas.
26 oct	Correcciones del perfil, enlace entre preguntas, detalles y perfiles, funciones de votos, imágenes en el detalle.

Fechas	Trabajo realizado
27 oct	Mejoras varias, cambio de imágenes a SVG, archivos adjuntos en respuestas y corrección de respuestas.
28 oct	Diseño adaptable (@media)
29 oct	Últimas mejoras que sugiere el profesor.
02-03 nov	Presentación y subida del servidor.

Recomendación: antes de entregar, ampliar el diagrama de Gantt con las tareas 4 a 10 y asignar responsables (ahora figuran como Team Member 1, 2 y 3), tal y como pide el enunciado: tareas, responsables, fechas y solapamientos.

3. Especificación de los requerimientos

Catálogo deducido del funcionamiento real de la aplicación. Conviene contrastarlo con el enunciado del reto.

3.1. Requerimientos funcionales

Código	Requerimiento	Rol
RF-01	Registro de usuario con nombre, apellidos, email, área de trabajo, antigüedad y contraseña. El email debe ser único.	Visitante
RF-02	Inicio de sesión con email y contraseña; mensaje de error si las credenciales no son válidas.	Visitante
RF-03	Cierre de sesión.	Usuario
RF-04	Consulta del perfil propio y del de otros usuarios: datos, foto, preguntas publicadas y estadísticas (preguntas, respuestas y votos recibidos).	Usuario
RF-05	Edición del perfil propio: nombre, apellidos, área, contacto, antigüedad y foto (máx. 2 MB).	Usuario
RF-06	Publicar una pregunta con título, descripción, tema y una imagen opcional.	Usuario
RF-07	Listar preguntas, buscar por título, filtrar por tema o por guardadas y ordenar (más recientes, más antiguas, más respuestas, menos respuestas).	Usuario
RF-08	Guardar y quitar de guardadas una pregunta; ver las guardadas en el perfil.	Usuario
RF-09	Ver el detalle de una pregunta con su autor, imagen y respuestas.	Usuario

Código	Requerimiento	Rol
RF-10	Responder a una pregunta o a otra respuesta, con hasta 5 archivos adjuntos.	Usuario
RF-11	Votar una respuesta (+1 / −1) y ver el total de votos sin recargar la página.	Usuario
RF-12	Borrar las propias respuestas.	Usuario
RF-13	Borrar cualquier pregunta o respuesta.	Administrador
RF-14	Listar usuarios con búsqueda y orden, ver su detalle, editar nombre, apellidos, área, email y rol, y eliminarlos.	Administrador

3.2. Requerimientos no funcionales

Código	Requerimiento
RNF-01	Arquitectura MVC en PHP con un enrutador único (index.php).
RNF-02	Acceso a datos con PDO y consultas preparadas.
RNF-03	Base de datos relacional MySQL/MariaDB con integridad referencial y borrado en cascada.
RNF-04	Interfaz adaptable (responsive) mediante reglas @media.
RNF-05	Validación en el cliente (JavaScript) y en el servidor (PHP).
RNF-06	Límites de subida: foto de perfil 2 MB; adjuntos de respuesta 10 MB cada uno y 5 por respuesta; tipos validados por contenido real (finfo).
RNF-07	Los archivos subidos se guardan fuera de la base de datos, en la carpeta uploads, con nombres aleatorios para preguntas y respuestas.
RNF-08	Control de acceso por sesión: solo usuarios identificados acceden a preguntas y detalles; las acciones de moderación requieren rol de administrador.
RNF-09	Usabilidad: iconos SVG, tipografía Archivo y mensajes de error junto a cada campo.
RNF-10	Código versionado en Git con ramas por persona integradas en develop.

3.3. Cumplimiento del enunciado

La aplicación cubre las seis funcionalidades mínimas del enunciado, con una duda sobre la votación de la respuesta favorita.

Funcionalidad del enunciado	Estado	Detalle
Registro y login de usuarios	Cubierto	RF-01, RF-02

Funcionalidad del enunciado	Estado	Detalle
Listado de preguntas y detalle	Cubierto	RF-07, RF-09
Publicación de preguntas (título, descripción, tema)	Cubierto	RF-06, con imagen opcional
Publicación de respuestas con adjuntos	Cubierto	RF-10, hasta 5 archivos de 10 MB
Votar respuesta favorita	Cubierto	Se vota +1 / −1 cada respuesta (RF-11). La tabla respuestaFavorita existe pero no se usa. Confirmar con el profesorado si basta con los votos.
Perfil de usuario	Cubierto	RF-04, RF-05
Extra: marcar pregunta como favorita	Cubierto	RF-08
Extra: perfil más completo	Cubierto	Historial de preguntas y guardadas
Extra: búsqueda avanzada	Cubierto	Por título, tema y orden.
Gestor de documentos	Cubierto	Organizados por categoria, nombre, autor o fecha.
Extra: notificaciones	No incluido	Mejoras futuras (apartado 6.3).

4. Especificación del diseño

4.1. Descripción de las funciones del software

Acceso y usuarios. Un visitante ve la pantalla de inicio de sesión (index.view.php) con acceso al registro. Al iniciar sesión se guarda el objeto de usuario en la sesión de PHP; en cada petición se comprueba que ese usuario sigue existiendo en la base de datos y, si no, se cierra la sesión. Los controladores InicioController, RegistroController y UsuarioController gestionan estas operaciones.

Preguntas. PreguntasController es el controlador por defecto. Muestra el listado y ofrece búsqueda por título (buscar), filtro por tema o favoritas con ordenación (buscarPorTema), guardado de preguntas (favorita, anadirFavoritos, eliminarFavoritos), detalle (detalle) y borrado por parte del administrador (eliminar). CrearPreguntaController valida el formulario, comprueba la imagen y la guarda en uploads/preguntas.

Respuestas y votos. Desde el detalle se puede responder (responder), borrar una respuesta (eliminarRespuesta) y votar (votar). Las respuestas admiten hilo mediante el campo idRespuesta. El voto se envía con Fetch API en formato JSON y el servidor devuelve el voto del usuario y el total actualizado. Volver a pulsar el mismo voto lo retira y no se permite pasar directamente de +1 a −1.

Administración. ListarUsuariosController y DetalleUsuarioController permiten buscar y ordenar usuarios, ver su ficha, cambiar su rol (admin o user) y eliminarlos. UsuarioController::destroy comprueba que quien borra es administrador.

Temas y áreas. Los temas de las preguntas son cinco: Ingeniería y Diseño, Normativa y Calidad, Mantenimiento, Sostenibilidad e Innovación, y Formación y Recursos. Las áreas de trabajo (12 en total) están definidas en config.php.

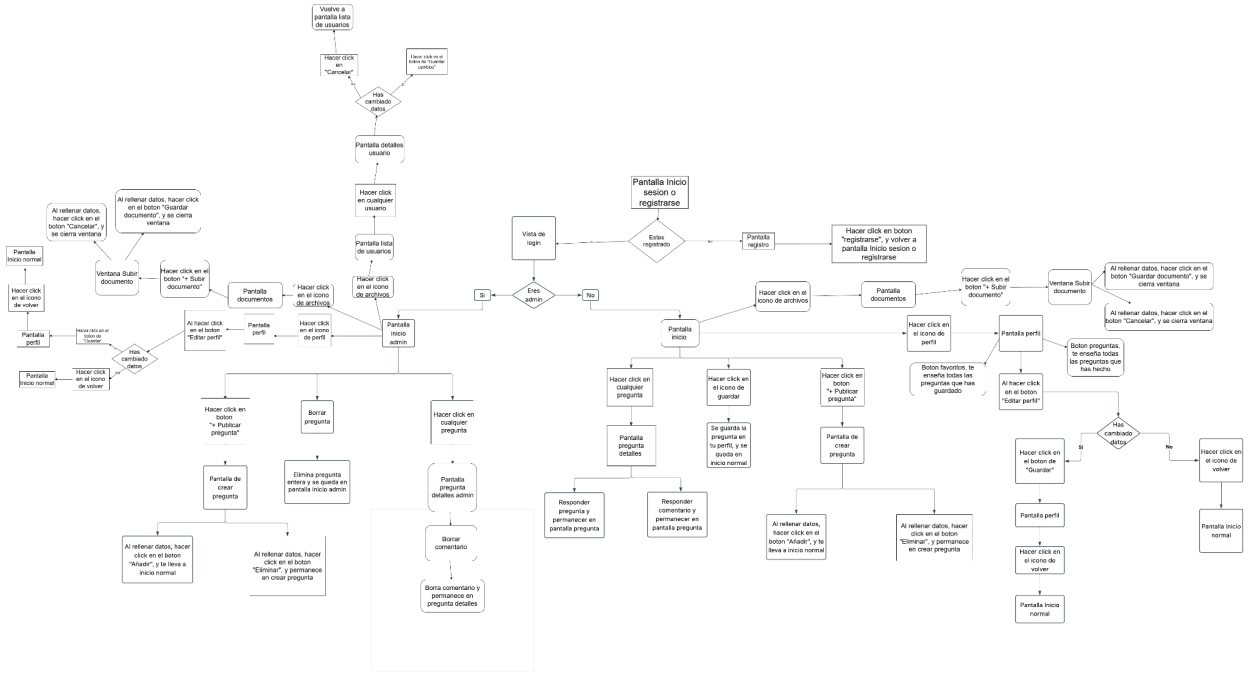
4.2. Descripción de la interfaz

La interfaz comparte cabecera (nav.php), pie (footer.php) y head (head.php) mediante la carpeta views/layout. Cada pantalla tiene su propia hoja de estilos y, cuando la necesita, su script.

Pantalla	Vista	Contenido principal
Inicio de sesión	index.view.php	Formulario con validación de email y contraseña, enlace al registro.
Registro	registro.view.php	Formulario con validación campo a campo.
Listado de preguntas	preguntas.view.php	Buscador, filtros por tema, selector de orden, tarjetas de pregunta y botón para crear.
Crear pregunta	crearPregunta.view.php	Título, descripción, tema, imagen opcional, botones Añadir y Borrar.
Detalle de pregunta	preguntas.detalle.view.php	Pregunta, imagen, respuestas con votos, formulario de respuesta con adjuntos.
Perfil	usuario.view.php	Datos, foto, estadísticas, preguntas propias y guardadas, edición del perfil.
Listado de documentos	documentos.view.php	Buscador, filtros por tema, selector de orden, lista de documentos subidos y botón para crear.
Listado de usuarios (admin)	listarUsuarios.view.php	Búsqueda, orden y acceso a cada ficha.
Detalle de usuario (admin)	detalleUsuarios.view.php	Edición de datos y rol, y borrado.

El flujo de navegación completo está en el diagrama Diagramas/Diagrama de navegacion.drawio.png. Resume dos recorridos: el del usuario normal (ver detalles, guardar, crear pregunta, responder, editar perfil, subir archivos) y el del administrador (los mismos más ver y administrar listado de usuarios, borrar

preguntas y comentarios).



4.3. Descripción de la arquitectura y el entorno tecnológico

La aplicación sigue el patrón **MVC** con un enrutador (front controller):

1. Toda petición llega a index.php, que inicia la sesión y llama a Router::dispatch().
2. El Router lee los parámetros controller y accion (por GET o POST) e invoca ese método. Por defecto usa PreguntasController e index.
3. El **controlador** valida los datos, llama al modelo y renderiza una vista (BaseController::render) o redirige.
4. El **modelo** encapsula las consultas SQL con PDO (clase Database con conexión única).
5. La **vista** es PHP + HTML con CSS y JavaScript propios; las acciones asíncronas (votos) usan Fetch API con JSON.
6. Los archivos (fotos y adjuntos) se almacenan en el sistema de ficheros (carpeta uploads) y su ruta se guarda en la base de datos.

Capa	Tecnología
Cliente	HTML5, CSS3 , JavaScript, Fetch API
Servidor	PHP con sesiones y PDO
Datos	MySQL/reto
Almacenamiento	Carpetas uploads/perfiles, uploads/preguntas y uploads/respuestas
Herramientas	Git y GitHub, draw.io (diagramas), diagrama de Gantt

Capa	Tecnología
Entorno de ejecución	Servidor local tipo XAMPP (Apache + MySQL) [COMPLETAR: confirmar versiones de PHP y MySQL]

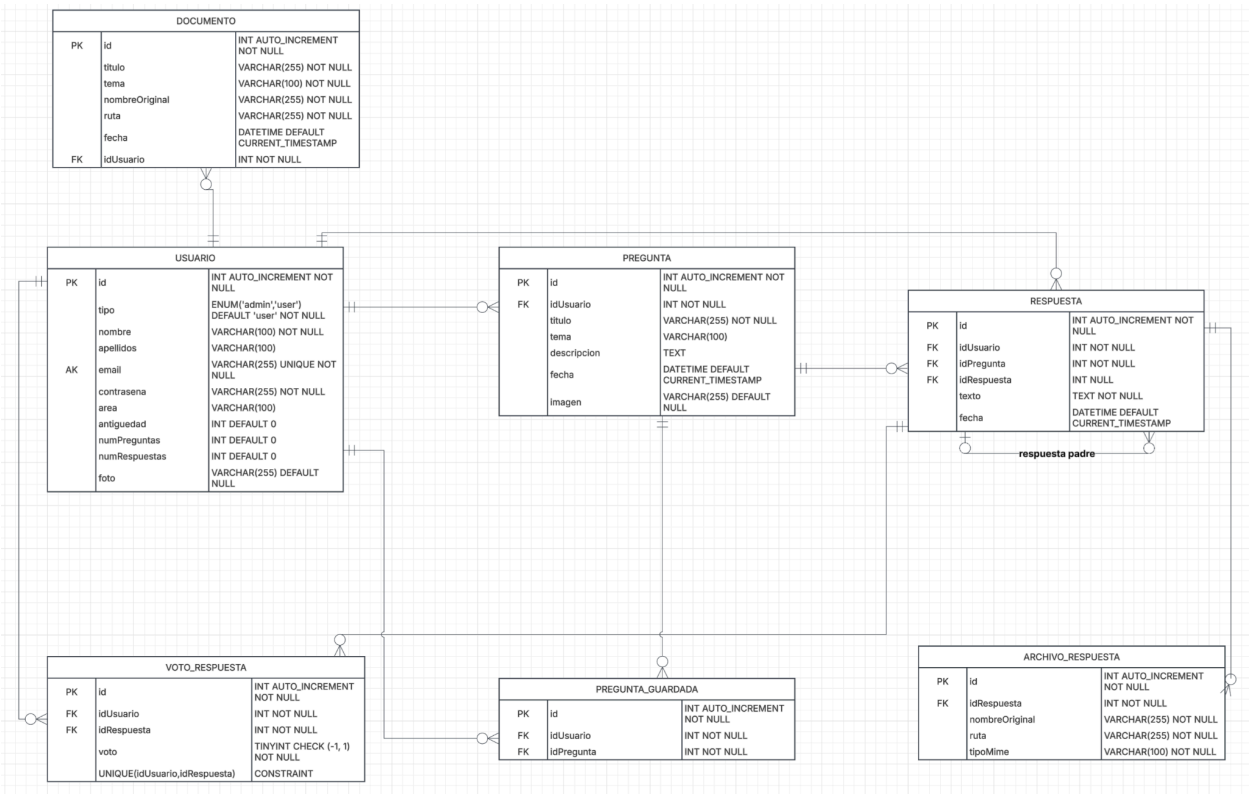
Estructura de carpetas: Codigo/ (index.php, Router.php, config/, controllers/, models/, views/, uploads/), BaseDatos/ (script SQL y MER), Diagramas/ y planificacion.gantt. El diagrama de arquitectura está en Diagramas/Diagrama de arquitectura.drawio.png.

4.4. Descripción del modelo de datos

El script BaseDatos/reto1Database.sql define 7 tablas:

Tabla	Campos principales	Relaciones
usuario	id, tipo (admin/user), nombre, apellidos, email (único), contrasena, area, antiguedad, numPreguntas, numRespuestas, foto	—
pregunta	id, idUsuario, titulo, tema, descripcion, fecha, imagen	Pertenece a un usuario
respuesta	id, idUsuario, idPregunta, idRespuesta, texto, fecha	Pertenece a un usuario y a una pregunta; idRespuesta apunta a otra respuesta (hilos)
votoRespuesta	id, idUsuario, idRespuesta, voto (−1 o +1)	Un voto por usuario y respuesta (restricción única)
preguntaGuardada	id, idUsuario, idPregunta	Preguntas guardadas por cada usuario
archivoRespuesta	id, idRespuesta, nombreOriginal, ruta, tipoMime	Adjuntos de cada respuesta
docuemnto	id, titulo, tema, nombreOriginal, ruta, fecha idUsuario	Pertenece a un usuario

Todas las claves foráneas usan ON DELETE CASCADE, de modo que al borrar un usuario, una pregunta o una respuesta se eliminan sus datos dependientes. El MER del repositorio (BaseDatos/MERDATABASE.png)



5. Consideraciones sobre la implementación

- **Validación doble.** Los formularios se validan en JavaScript con expresiones regulares (nombre con inicial mayúscula, apellidos de dos palabras, email, antigüedad numérica, contraseña con mínimo de 8 caracteres, mayúscula, minúscula, número y símbolo) y de nuevo en PHP.
- **Subida de archivos segura.** Se comprueba el tipo MIME real con finfo, el tamaño y el número de archivos. Las imágenes de preguntas y adjuntos reciben nombre aleatorio (bin2hex(random_bytes)). La foto de perfil se guarda como usuario_ID.ext.
- **Consultas preparadas.** Todas las consultas usan parámetros de PDO.
- **Votos asíncronos.** La restricción única (idUsuario, idRespuesta) y la comprobación CHECK (voto IN (−1, 1)) garantizan la integridad; la interfaz se actualiza sin recargar.
- **Borrado en cascada.** Reduce el código necesario para mantener la consistencia, aunque PreguntasModel::deleteById también limpia manualmente las dependencias.
- **Sesión.** Al inicio de cada petición se revalida el usuario contra la base de datos.
- **Trabajo en equipo.** Cada integrante trabaja en una rama propia (por ejemplo ramaSara) y la integra con la rama develop mediante pull frecuentes.
- **Diseño adaptable.** Hay 19 reglas @media repartidas por las hojas de estilo.

Instalación y datos de prueba

1. Copiar la carpeta Codigo/ en el directorio del servidor web (por ejemplo htdocs).
2. Crear la base de datos reto e importar BaseDatos/reto1Database.sql (ver incidencia 1 de la sección 7).
3. Revisar Codigo/config/config.php (host localhost, base reto, usuario root, sin contraseña).

4. Asegurar permisos de escritura en uploads/perfiles, uploads/preguntas y uploads/respuestas.
5. Abrir http://localhost/reto1/index.php.

Rol	Email	Contraseña
Administrador	admin@gmail.com	Aa-12345
Usuario	user@gmail.com	Uu-12345

6. Incidencias del proyecto y conclusiones

6.1. Incidencias detectadas en la revisión

Nº	Incidencia	Impacto	Propuesta
1	Las contraseñas se guardan y se comparan en texto plano.	Riesgo de seguridad si se filtra la base de datos.	Usar password_hash al registrar y password_verify al iniciar sesión, ampliando los datos de prueba.
2	El Router carga cualquier controlador y ejecuta cualquier método indicado por la URL, sin lista blanca.	Se pueden invocar métodos no previstos.	Validar controlador y acción contra una lista de permitidos.
3	El borrado de usuarios, preguntas y respuestas se hace con peticiones GET en algunos casos.	Borrados accidentales o forzados desde enlaces.	Usar POST con confirmación.

6.2. Incidencias de proceso

- El historial muestra pulls frecuentes desde develop, lo que redujo los conflictos.
- Cambios de última hora (imágenes a SVG, archivos en respuestas, diseño adaptable y gestión de archivos) concentraron mucho trabajo en los últimos días.

6.3. Conclusiones

Se ha construido un foro interno funcional que cubre el ciclo completo: registro, publicación de preguntas, respuestas con adjuntos, votos, guardado, gestor de documentos, perfiles y moderación por parte del administrador. La separación en modelos, vistas y controladores, el uso de consultas preparadas y la validación de archivos aportan una base sólida y mantenible.

Como mejoras futuras destacan: cifrar las contraseñas, notificaciones de nuevas respuestas, marcar una respuesta como solución.